

Closing the Cross-Dialect Gap: Query Plans as a Portable Interface in Text-to-SQL

Corentin Royer^{1,2}, Robin Oester¹, Yotam Perlitz¹, Yannick Metz²,
Andrea Giovannini¹, Mennatallah El-Assady²

¹IBM Research, Zurich, Switzerland ²ETH Zurich, Zurich, Switzerland

Correspondence: corentin.royer@ibm.com

Abstract

Text-to-SQL systems are typically trained and evaluated on a single dialect (SQLite), yet production deployments span PostgreSQL, MySQL, ClickHouse, and beyond. We show that this single-dialect assumption leads to a substantial drop in lower-bound dialect accuracy for every model we tested. The drop persists across scale, architecture, and even purpose-built text-to-Structured Query Language (SQL) systems. We argue that the fix is to change the generation target: instead of asking a large language model (LLM) to emit dialect-specific SQL, we have it emit a dialect-agnostic relational algebra query plan, which a deterministic compiler then renders into SQL for any supported backend. Across thirteen models from 3B to frontier scale, this restores cross-dialect portability nearly uniformly, at a small cost in peak accuracy on the model’s home dialect for capable prompted models and none once fine-tuned on plans; under matched fine-tuning, plan supervision yields a stronger model than SQL supervision. We also introduce DF-Match, a question-aware result-set comparator needed to evaluate fairly across dialects, where existing metrics confound semantic errors with benign cross-dialect variation. More broadly, the result is a reminder that a generation target chosen for *execution* is not necessarily the one that maximizes *generation quality*.

1 Introduction

Natural-language interfaces to databases have advanced rapidly, with LLMs now achieving over 70% execution accuracy on standard text-to-SQL benchmarks (Li et al., 2025; Gao et al., 2024; Talaie et al., 2024). These results suggest that LLM-based query generation is approaching practical utility. Yet virtually all existing systems share a critical assumption: they target a single SQL dialect (typically SQLite), the language of the dominant benchmarks (Yu et al., 2018; Li et al., 2024b). In

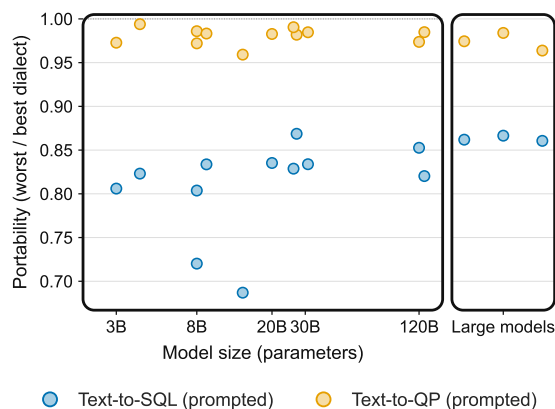


Figure 1: **Query plans restore cross-dialect portability at every scale.** Portability (lower-bound / best-dialect accuracy ratio over {SQLite, PostgreSQL, MySQL, ClickHouse}; higher is better) vs. model size on BIRD. The query plan cloud sits at ≥ 0.96 for all sizes; the SQL cloud is lower and only mildly responsive to scale. Open-weight models on the log-scale left panel; closed-weight frontier models on the right.

production, however, organizations rarely operate a single database engine. Cloud deployments routinely span PostgreSQL, MySQL, and other backends, thus a text-to-SQL system that works only on SQLite is of limited practical value.

The cost of this assumption is high. When we evaluate models trained on SQLite across multiple backends, we observe poor portability universally: across every model we evaluate, *lower-bound dialect accuracy*, the minimum execution accuracy over the four dialects we evaluate (which is a guarantee on accuracy regardless of the deployed dialect), land 8–17 percentage points below SQLite on BIRD. This degradation is not limited to a single model or architecture; it persists across fine-tuned and prompted models alike, including purpose-built text-to-SQL systems. The root cause is that SQL dialects differ in function signatures (strftime vs. to_char), type systems (implicit vs.

explicit casting), regex syntax, and NULL-handling semantics. An LLM trained to produce SQLite inevitably generates dialect-specific syntax.

We propose *text-to-query plan* as a portable alternative to text-to-SQL: instead of emitting dialect-specific SQL, the LLM generates a single Calcite-compatible (Begoli et al., 2018) query plan (a tree of operators such as scans, filters, joins, projections, and aggregations, that represents the logical structure of a query without committing to any dialect’s syntax), which a deterministic compiler then translates into executable SQL for the target backend. Because dialect adaptation is handled mechanically rather than by the model, performance across dialects is more reliable. We make the following contributions:

1. **A dialect-agnostic conversion pipeline** between SQL and Calcite-compatible query plans, with 94–97% round-trip fidelity across dialects.
2. **DF-Match**, an ordering- and tie-aware cross-dialect comparator that improves human agreement over BIRD execution accuracy (BIRD-EA).
3. **An empirical validation of query plans as a multi-backend generation target**, in four parts:
 - (a) Plans restore portability uniformly across scales: the gap in lower-bound dialect accuracy relative to SQLite shrinks from 8–17 pp to at most 2.3 pp on BIRD (§6.3).
 - (b) For sufficiently capable LLMs ($\gtrsim$ 68B), plans match or exceed SQL on lower-bound dialect zero-shot accuracy on BIRD, without query plan-specific training (§6.4).
 - (c) Under matched fine-tuning, plan supervision beats SQL supervision on accuracy and schema grounding (§6.5).
 - (d) Plans show no consistent structural disadvantage: across 13 models, no query-level feature consistently favors one representation (§6.6).

We report lower-bound accuracy throughout, since it is what a practitioner can rely on once the backend is fixed: by that measure the switch is close to free, costing capable prompted models a few points of peak accuracy on their home

dialect (§6.4) and nothing once fine-tuned. The takeaway behind these four claims is the same: text-to-SQL adopted SQL as a target because SQL is what databases execute, but a target chosen for *execution* is not necessarily the one that maximizes *generation quality or portability*. Whenever the executable surface (SQL, in our case) serializes a structured representation (a relational algebra plan), generating the structure and rendering the surface deterministically is a viable alternative.¹

2 Related Work

Text-to-SQL and cross-dialect portability. Text-to-SQL has shifted from rule-based parsers (Zelle and Mooney, 1996) and schema-aware encoders such as RAT-SQL (Wang et al., 2020) to LLM-based generation, with prompting pipelines like DAIL-SQL (Gao et al., 2024), DIN-SQL (Pourreza and Rafiei, 2024), and CHESS (Talaie et al., 2024), and open-source models trained on large-scale synthetic data (Li et al., 2024a, 2025). Nearly all of this work targets a single dialect: SQLite, the language of Spider (Yu et al., 2018) and BIRD (Li et al., 2024b). PolySQL (Perlitz et al., 2026) first measured the resulting lack of portability systematically, uncovering drops in lower-bound dialect accuracy of up to 10 pp compared to SQLite. Subsequent work targets the same problem but stays at the SQL level: SQL-GEN (Pourreza et al., 2024) synthesizes per-dialect training data and merges expert models, MoMQ (Lin et al., 2024) uses a mixture of dialect experts, and ExeSQL (Zhang et al., 2025) bootstraps dialect competence through execution-driven training. These approaches adapt the *model* to each dialect, so the adaptation is repeated for every new dialect and every new model. We instead replace SQL with a dialect-agnostic plan and delegate dialect adaptation to a deterministic compiler, which is adapted once and reused across every model we evaluate.

Intermediate representations as generation targets. Generating a structured Intermediate Representation (IR) rather than surface code has a long lineage, from AST-based semantic parsing (Dong and Lapata, 2016; Rabinovich et al., 2017; Yin and Neubig, 2018) to constrained decoding over SQL grammars (Scholak et al., 2021; Geng et al., 2023). In the text-to-SQL setting, IRNet’s SemQL (Guo

¹Code is available at <https://ibm.biz/~tyrUnBW0j>.

et al., 2019), QPL (Eyal et al., 2023), and Query-Gym (Ananthakrishnan et al., 2026) propose custom intermediates, and Dial (Zhang et al., 2026) pairs a dialect-aware logical-plan module with execution-driven verification. Closest to our setting, Dialect-SQL (Shi et al., 2025) also generates an intermediate representation that is then translated into each target dialect, though it assumes this translation is lossless rather than measuring it. These intermediates are custom DSLs designed for a single pipeline and a single dialect (typically SQLite). The database community has independently developed cross-engine algebra formats with the same dialect-agnostic, algebra-level abstraction: Apache Calcite (Begoli et al., 2018) and Substrait (substrait.io, 2021), a cross-language serialization format designed to decouple query producers from execution engines. Our target is Calcite’s algebra IR, chosen so that one query plan compiles to multiple production backends. Where prior work shows an IR can beat SQL baselines within one pipeline, we instead ask whether the IR is a sound generation target in its own right.

Evaluation metrics. Early text-to-SQL benchmarks scored predictions by comparing the SQL itself, e.g. Spider’s exact set match over query clauses (Yu et al., 2018), which penalizes semantically equivalent rewrites. Executing queries and comparing result sets sidesteps this, and BIRD execution accuracy (Li et al., 2024b), an unordered row-set comparison (ignoring order, duplicates, and cardinality), has become the de facto standard, including on enterprise-backend benchmarks like Spider 2.0 (Lei et al., 2025). However, Result-based comparison has its own failure modes: false positives in single-instance databases (Zhong et al., 2020), and structural false negatives whenever ordering or column-shape conventions diverge from the gold query (23% FP, 29% FN under ETM (Ascoli et al., 2024)), both amplified across dialects, where row ordering and type semantics diverge. LLM-based judges (Kim et al., 2024; Zheng et al., 2023) sidestep these structural mismatches by comparing at a higher level of abstraction, but inherit known biases, including self-preference (Panickssery et al., 2024).

3 Preliminaries

This section sets up the three concepts this paper builds upon: the text-to-SQL task, BIRD-EA as the baseline result-equivalence metric, and the query

plan itself as a structured generation target.

Text-to-SQL Task. Let D be a relational database, let q be a natural-language question, and let p^* be a gold SQL query that answers q on D . Let $\text{df}(p, D)$ denote the result set obtained by executing query p on D , and let $\text{eq}(\cdot, \cdot)$ be a comparison function over result sets (specified below). Given q and D , the text-to-SQL task is to generate a query $\hat{p}$ such that $\text{eq}(\text{df}(\hat{p}, D), \text{df}(p^*, D)) = 1$.

BIRD Execution Accuracy. A standard choice for $\text{eq}(\cdot, \cdot)$ is BIRD-EA (Li et al., 2024b), which deems two dataframes equal whenever their rows form the same set, ignoring row order and duplicate counts but remaining sensitive to column shape: $\text{eq}_{\text{BIRD-EA}}(\hat{T}, T^*) = 1$ iff $\text{set}(\hat{T}) = \text{set}(T^*)$. This is the default metric on BIRD and most downstream benchmarks.

Query Plans. A query plan $\hat{\pi}$ is an *algebraic* specification of a query: a tree whose nodes are operations on dataframes, where leaves are TableScan nodes that load a base table, and internal nodes (Filter, Project, Join, Aggregate, Sort) transform the dataframes produced by their children (Figure 2). In contrast to SQL, which declares *what* result is desired, a plan prescribes *how* it is computed. A deterministic compiler converts $\hat{\pi}$ to an executable query $\hat{p}$ in any supported dialect (Section 4.1).

4 Method

Studying query plans as a multi-backend generation target requires two enablers that the existing text-to-SQL ecosystem does not provide: a faithful converter between SQL and query plans, so that plans can be used as a generation target (§4.1); and a result-set metric that tolerates the cross-dialect variation that yield errors with BIRD-EA, so that any generation target can be evaluated fairly across backends (§4.2). Figure 2 shows how the two fit together across the four stages of the workflow, and in particular where the conversion pipeline is used in each direction.

4.1 Conversion pipeline

The pipeline runs in both directions. The *forward* direction ($\text{SQL} \rightarrow \text{plan}$) converts gold SQLite queries into reference plans π^* . The *backward* direction ($\text{plan} \rightarrow \text{SQL}$) compiles any plan $\hat{\pi}$ into executable SQL for a target backend E . Both directions are built on Apache Calcite (Begoli et al.,

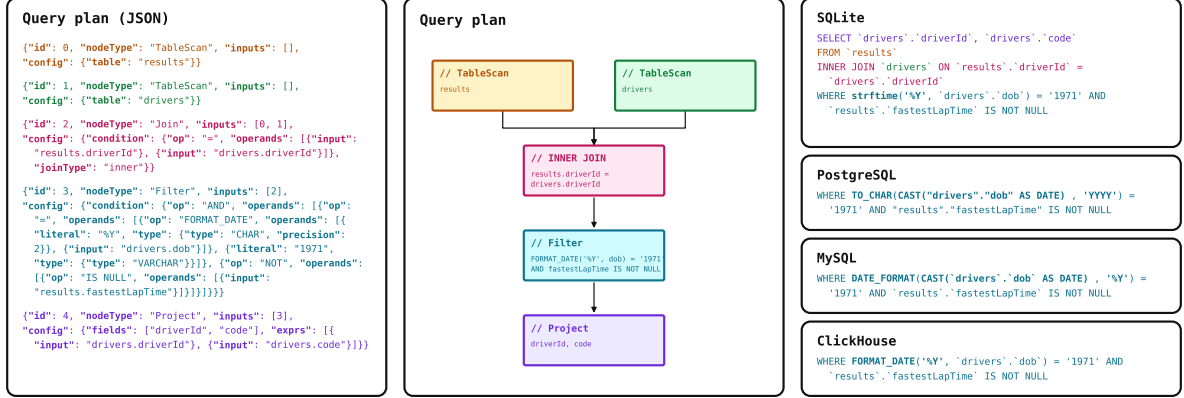


Figure 2: **Evaluation pipeline, and one query plan in four SQL dialects.** *Top:* the LLM emits a *single* query plan; the compiler renders it to SQL for every backend; each query executes on its own dialect-parallel copy of the database (§5.2); and DF-Match compares each predicted dataframe against the gold dataframe from the gold SQLite query. *Bottom:* that same plan shown as emitted JSON (left), as an operator tree (centre), and compiled to the four dialects (right). Colored anchors link each operator to the SQL fragment it generates. Dialect-specific details, such as function renames (strftime/TO_CHAR/DATE_FORMAT) and identifier quoting, sit *inside* the colored spans: the model emits the plan once and never has to choose between them.

2018): the IR is a JSON serialization of a Calcite logical query plan, which the forward direction produces and the backward direction consumes.

We measure the fidelity of the pipeline by *round-trip equivalence*: starting from a SQL query p , the forward direction produces a plan π , and the backward direction recompiles π to a SQL query $\hat{p}_E$ on backend E . The round-trip succeeds when the recompiled query yields the same result as the original, i.e. $\text{eq}(\text{df}(p, D), \text{df}(\hat{p}_E, D)) = 1$. Section 6.1 reports this rate per backend on BIRD and Spider.

Forward: SQL $\rightarrow$ plan. Benchmarks ship gold queries in SQLite, but Calcite’s SQL parser does not accept SQLite syntax. We therefore convert each gold query p^* in three steps:

(1) *SQLite $\rightarrow$ MySQL transpile* using SQL-Glot (Mao, 2023). We target MySQL because its syntax is closest to SQLite, minimizing the transpilation errors.

(2) *MySQL $\rightarrow$ Calcite plan*: Calcite parses the rewritten query into a logical operator tree. We disable Calcite’s optimization rules so the plan shape mirrors the original query; otherwise, rewrites such as predicate pushdown, or stripping IS NOT NULL checks on columns Calcite considers non-nullable, would alter what the LLM has to learn and break portability to backends with different nullability semantics.

(3) *Serialize* the tree to a compact JSON format (one object per operator; Figure 2) with symbolic column references so the encoding is robust across

reorderings, yielding π^* .

Backward: plan $\rightarrow$ SQL. Given a plan π and a target backend $E \in \{\text{SQLite}, \text{PostgreSQL}, \text{MySQL}, \text{ClickHouse}\}$, we deserialize π into a Calcite logical plan and emit SQL through a *custom Calcite dialect* we implement for each backend. Our dialects perform two kinds of adaptation. *Syntactic*: identifier quoting, LIMIT/OFFSET layout, function renaming such as $\text{FORMAT_DATE} \rightarrow \text{STRFTIME/TO_CHAR/DATE_FORMAT}$. *Semantic*: aligning behavior across backends. Examples include: enforcing SQLite’s NULL-first sort order when emitting for PostgreSQL; rewriting LIKE to ILIKE to preserve SQLite’s case-insensitive match on PostgreSQL; wrapping string-to-integer casts to mimic SQLite’s lenient conversion where PostgreSQL would raise; and keeping textual comparisons textual instead of letting PostgreSQL’s stricter typing coerce them to numeric.

Concretely, instantiating round-trip equivalence with $\text{eq} = \text{eq}_{\text{BIRD-EA}}$ yields a large number of spurious failures driven by cross-dialect row ordering and tie-breaking rather than real semantic drift, motivating the ordering- and tie-aware comparator we introduce next.

4.2 The DF-Match framework

BIRD-EA has two main failure modes even in single-dialect evaluation, with high false-positive and false-negative rates (Ascoli et al., 2024; Kim

et al., 2024); cross-dialect evaluation compounds both, as we show empirically in §6.2. Consider two questions where BIRD-EA fails in opposite directions. (a) “List the top three clients by spending, in order.” A model that returns the correct three clients in the wrong order is accepted by BIRD-EA because $\text{set}(\text{predicted}) == \text{set}(\text{gold})$ ignores row order (*false positive*). (b) “Which client spent the most?” with gold query `ORDER BY total DESC LIMIT 1`. If two clients are tied at the top, the gold reference contains whichever one the backend returned first; a model that returns the other tied client is rejected by BIRD-EA (*false negative*).

DF-Match addresses both by deciding, *per question*, what counts as a correct result. It (i) classifies each question along two axes that determine which constraints to enforce, (ii) extracts tie information from the gold SQL so that semantically equivalent rows can be permuted, and (iii) compares the predicted and gold dataframes under per-class rules built from the previous two ingredients.

Question classification. Each question is labeled along two axes: **ordering sensitivity**, whether the question asks for a ranked result (“rank...”, “top five...”); and **row-count sensitivity**, whether it pins the output cardinality (“the highest-spending client” $\rightarrow 1$). We classify the *natural-language question*, not the gold SQL, because the gold query is sometimes under-specified whereas the question carries the intent. The two booleans are produced by an LLM classifier (validated in §6.2; full protocol in Appendix F). Four combinations are possible, but the unordered-fixed cell is empty in practice, leaving three classes: `not_ordered`, `ordered_fixed`, and `ordered_not_fixed`.

Tie handling. Once a question is identified as ordered, the comparison rules need to know which result rows are *tied* under the gold ordering before permitting legitimate reorderings, and which rows the backend could legitimately have placed at a LIMIT boundary. DF-Match derives both by AST-rewriting the gold SQL into two auxiliary queries: the *order-key query*, which recovers the sort values per row so that consecutive equal values define a *tie group*, and the *boundary-alternatives query*, which enumerates rows the gold could have placed at a LIMIT cut. Both are executed once per gold query and cached; the next paragraph shows them in use.

Worked example. Returning to the two questions from the top of this section. Question (a),

“List the top three clients by spending, in order,” is classified `ordered_fixed`. From a gold query `SELECT name FROM clients ORDER BY total_spend DESC LIMIT 3`, the order-key query `SELECT total_spend FROM clients ORDER BY total_spend DESC` returns the sort values; consecutive equal values define tie groups. The comparison enforces positional row-by-row equality, but accepts any permutation of rows whose positions fall in the same tie group: a prediction that swaps two clients tied at the same `total_spend` is accepted; a prediction that swaps two clients with distinct spends is rejected. Question (b), “Which client spent the most?” (gold `ORDER BY total_spend DESC LIMIT 1`), is classified `ordered_not_fixed`. With $K = 1$ and a tie at the top (say at `total_spend = 90`), the boundary-alternatives query becomes `SELECT name FROM clients WHERE total_spend = 90`, enumerating every client the backend could have placed at the cut; any of them is accepted, addressing the BIRD-EA false negative.

Per-class comparison. Comparing predicted $\hat{T}$ to gold T^* first maps gold columns to predicted columns by matching distinct value sets (tolerating duplicate-count differences from `DISTINCT/GROUP BY` divergences); a gold column with no match rejects $\hat{T}$. The class-specific row check then proceeds:

- `not_ordered`: row multisets must match, tolerating duplicate-count differences from `DISTINCT/GROUP BY` divergences.
- `ordered_fixed`: cardinalities must be equal and rows must match positionally, modulo permutations within tie groups and consecutive duplicates.
- `ordered_not_fixed`: $|\hat{T}| \geq |T^*|$, the predicted prefix must match the gold as in `ordered_fixed`, with boundary alternatives accepted when the cut falls inside a tie group.

Numeric comparisons use $\varepsilon = 10^{-9}$ throughout. Pseudocode for the per-class dispatch is in Appendix G.

Relation to BIRD-EA. BIRD-EA is a permissive instance of the `not_ordered` branch: it compares tuple sets, so row order, duplicate counts, and cardinality are all ignored. This exposes it to both false positives and false negatives. DF-Match is

designed so that, given a correct question classification (validated in §6.2), it never produces false positives (sketch in Appendix H): its accuracy is therefore a *lower bound* on true accuracy, whereas BIRD-EA is neither a lower nor an upper bound.

5 Experimental setup

We evaluate query plan generation against direct SQL generation across four dialects, using a diverse set of models and a controlled fine-tuning comparison. This section describes the benchmarks (§5.1), the cross-dialect evaluation environment (§5.2), the models we compare (§5.3), our training setup (§5.4), and the error-categorization procedure used in our analysis (§5.5).

5.1 Benchmarks & Datasets

We evaluate on the dev splits of two standard text-to-SQL benchmarks: BIRD (Li et al., 2024b) and Spider (Yu et al., 2018). Both ship with SQLite databases, gold SQLite queries, and natural-language questions spanning multiple domains. We use the dev splits because the test sets are not publicly available. For fine-tuning (§5.4), we additionally draw training data from SynSQL (Li et al., 2025), a large-scale synthetic text-to-SQL corpus. SynSQL is used only as a training source and is not part of our evaluation. Licenses and intended-use details for all three datasets are in Appendix E.

5.2 Dual Execution

Cross-dialect evaluation requires the same question executed against multiple backends. We follow Perlitz et al. (2026) and migrate every SQLite database in BIRD and Spider to PostgreSQL, MySQL, and ClickHouse via schema-level ETL using the DLT data-loading library (dltHub, 2022), yielding four dialect-parallel copies with identical schemas, contents, and questions. Each predicted query runs against the corresponding backend and is evaluated with DF-Match (§4.2).

5.3 Models & Baselines

Each model is evaluated in two regimes. The SQL regime prompts the model once per target dialect (four runs over SQLite, PostgreSQL, MySQL, ClickHouse) with a one-shot example differing only in the target-dialect name. The query plan regime prompts the model once with a one-shot example plus a brief specification of the plan language (operator set and syntax), and our pipeline (§4.1) compiles the emitted plan to executable SQL

for all four dialects. The model set spans three size tiers and mixes open-weights, API-served, dense, and mixture-of-experts architectures (full list with parameter counts in Table 2); we additionally include two purpose-built text-to-SQL systems, OmniSQL (Li et al., 2025) and SQLCoder², as a reference point for how SQLite-specialized models generalize off-dialect. All prompted runs use greedy decoding.

5.4 Training

We fine-tune Ministral-3-8B on a 1M-example mixture drawn from SynSQL (Li et al., 2025) and the BIRD-train split, comparing two supervision targets: the gold SQL query p^* , or a gold plan π^* obtained by passing p^* through the SQL→Calcite pipeline (§4.1). Examples where the pipeline fails are excluded from *both* runs, so the SQL and query plan fine-tunes see identical questions and differ only in the generation target. We train with LoRA (Hu et al., 2022); optimizer, learning-rate schedule, and compute budget are held constant across runs. Full hyperparameters are in Appendix C.

5.5 Error Categorization

To understand what kinds of errors each generation target produces, we classify each error with an LLM judge (Claude Haiku 4.5, temperature 0) into one of six categories: Schema Linking, Filtering/Logic, Aggregation/Grouping, Ordering/Ranking, Dialect Syntax, or Other. The judge sees the question, gold and predicted SQLite queries, and any execution error; query plans are compiled to SQLite first so the judge sees the same input distribution regardless of generation target. Category definitions and the user-prompt template are in Appendix D. Hand-checking a sample of its labels against manual annotation gives 76% precision across the six categories. The judge only sorts errors that have already been detected by DF-Match into buckets, so its accuracy shapes the breakdown in §6.6 but does not enter any headline metric.

6 Results

We organize this section to mirror the contributions: §6.1 and §6.2 first validate the two instruments (the conversion pipeline and DF-Match), §6.3–§6.6 then address the four claims about query plans as a generation target.

²<https://huggingface.co/defog/sqlcoder-7b-2>

	SQLite	PG	MySQL	CH
BIRD	97.1	95.4	95.6	94.7
Spider	97.2	96.7	96.6	95.6

Table 1: Round-trip conversion accuracy (%) of the pipeline under DF-Match. PG = PostgreSQL, CH = ClickHouse. Each gold SQLite query is converted to a Calcite plan and back to the target dialect; accuracy measures whether the reconstructed query returns an equivalent result as the original.

6.1 The conversion pipeline is reliable

Table 1 reports round-trip accuracy of the conversion pipeline under DF-Match on BIRD and Spider across the four backends. The pipeline achieves 94–97% fidelity everywhere. The remaining 3–6% are dominated by errors in the gold query itself and by cross-backend typing mismatches that surface only after compilation (for example, implicit string-to-numeric coercions that SQLite tolerates but PostgreSQL or ClickHouse reject). These queries are retained at evaluation time: a round-trip failure on the gold query does not exclude a model from generating a different but semantically correct query plan that answers the question, so excluding them would bias the benchmark toward plan-convertible questions.

6.2 DF-Match agrees with human judgment

Before reporting results under DF-Match, we validate its performance. The LLM classifier that drives DF-Match’s per-class comparison reaches **93.4%** balanced accuracy (mean of per-class recall) on the ordering-sensitivity axis and **99.5%** on the row-count-sensitivity axis, measured on 103 hand-annotated BIRD questions. Applied to the full BIRD set, the classifier labels 86.6% of questions as `not_ordered`, 11.9% as `ordered_fixed`, and 1.5% as `ordered_not_fixed`. On a separate sample of 103 cases where DF-Match and BIRD-EA return opposite verdicts, a human annotator sides with DF-Match in 96.1% of cases. Replacing BIRD-EA with DF-Match also raises apparent round-trip pipeline reliability by 14–16 pp per dialect. Full protocols, per-axis numbers, the BIRD-EA comparison table, and study limitations are in Appendix F.

6.3 Query plans restore portability

Figure 1 plots portability on BIRD against model size for every model we evaluate. The pattern is the same at every scale and architecture: SQL tar-

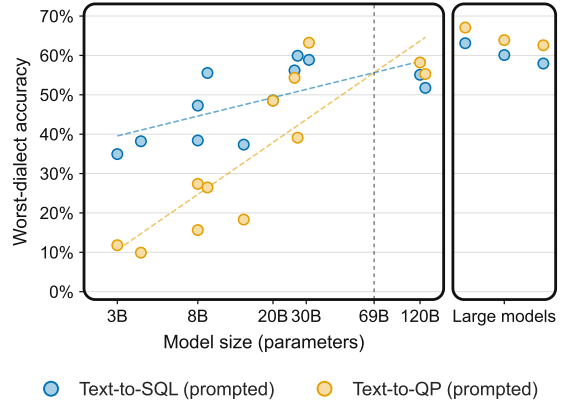


Figure 3: **Plans win above ~ 68 B.** Lower-bound dialect accuracy on BIRD vs. model size: per model, the lowest accuracy across the four dialects. The query plan trend rises more steeply than SQL; the two cross at ~ 68 B (dashed line), beyond which plans deliver as much or more lower-bound dialect accuracy than SQL. Per-model breakdown in Table 2.

gets land at 0.69–0.87 portability with lower-bound accuracy drops of 8–17 pp, while query plan targets land at ≥ 0.96 with lower-bound accuracy drops at most 2.3 pp. Neither cloud trends with model size: scaling does not buy SQL portability. The drop is driven by ClickHouse and PostgreSQL; MySQL accuracy tracks SQLite for most models. Per-dialect numbers for every model are in Appendix A (Table 2), and the same trend holds on Spider (Appendix B). Whether this comes at a cost in peak accuracy on the model’s home dialect is the question we address next.

6.4 Capable models produce query plans zero-shot

Figure 3 reports lower-bound dialect accuracy on BIRD as a function of model size. The query plan trend rises more steeply with scale than the SQL trend, and the two cross around 68B parameters. We therefore use *capable* operationally to mean prompted models above approximately 68B parameters: above this threshold, query plans deliver at least as much lower-bound dialect accuracy as SQL on BIRD, with no query plan-specific training and from a single format demonstration. On Spider, where the SQL baseline is already close to its ceiling, the two targets are instead near parity for these models. Below it, the query plan regime is strictly harder for the model: across the small and mid prompted band, plan-format errors drag lower-bound dialect accuracy 19–29 pp below the SQL

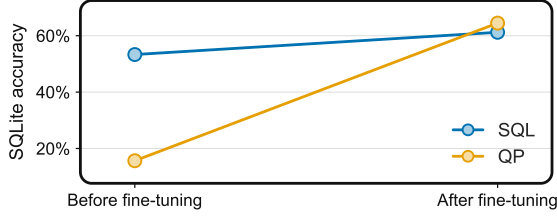


Figure 4: **At matched cost, plans yield a stronger model.** SQLite accuracy on BIRD for Ministral-3-8B before and after fine-tuning, on identical data. Prompted SQL starts well above prompted query plan, but query plan supervision overtakes SQL supervision after fine-tuning.

baseline, and the cross-over has not yet occurred.

6.5 Query plans are a strong training target

Holding training data, optimizer, and compute identical (§5.4), we fine-tune Ministral-3-8B twice, once on gold SQL, once on gold query plans, and evaluate both on BIRD. The query plan-supervised model reaches **64.5%** on SQLite versus **61.2%** for the SQL-supervised model, while keeping the lower-bound accuracy drop at 1.7 pp (portability 0.97 vs. 0.82 for the SQL fine-tune; Table 2). Query plans therefore do more than inherit the SQL baseline: at matched cost, they produce a *better* model. This is despite a substantially lower starting point: prompted Ministral-3-8B reaches 53.3% under SQL but only 15.7% under query plans, so query plan supervision lifts the model by 48.8 pp against the 7.9 pp gain from SQL supervision.

6.6 Query plans have no consistent structural disadvantage

If query plans carried an intrinsic cost, it would surface either as a *category* of errors elevated under query plans, or as a query-level *structural feature* that predicts query plan losses. We check both and find no such signal.

Reasoning errors are tied; the surplus is a decoding tax. Across the 5 strongest prompted models (Figure 5a), the reasoning categories (Schema Linking, Filtering/Logic, Aggregation/Grouping) are essentially matched between SQL and query plan. The only query plan surplus concentrates in Dialect Syntax, and inspection shows these are overwhelmingly malformed-JSON outputs: a decoding-level tax that constrained decoding or resampling could plausibly address, and one that largely collapses

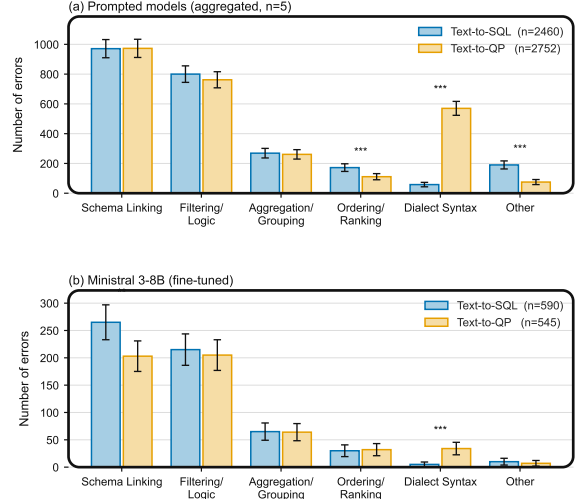


Figure 5: **No reasoning-error category penalizes plans.** LLM-classified error counts on BIRD. (a) the 5 strongest prompted models aggregated (3 frontier models, GPT-OSS 120B, Mistral Medium); (b) fine-tuned Ministral-3-8B. Reasoning categories are near-identical between SQL and query plan. The only query plan surplus is Dialect Syntax (panel a), which vanishes after fine-tuning (panel b).

after LoRA fine-tuning at 8B (Figure 5b). The surplus is also a property of plan *generation*, not of compilation: separating failures that occur on all four backends (a wrong plan) from those on a single backend (a compilation defect), compilation accounts for only 2–3% of a dialect’s failures on BIRD and 5–7% on Spider, consistent with the round-trip fidelity on gold plans in §4.1 (94–97%).

Sort and limit errors drop under query plans.

A small structural *advantage* of the query plan format is that ordering and cardinality are explicit operators (SORT, AGGREGATE) rather than parsed clauses. Empirically, Ordering/Ranking errors drop from 172 under SQL to 109 under query plan on the same 5 models, a 37% reduction in the one category where the surface forms differ most.

No query-level feature consistently favors one representation.

A structural cost might also hide in queries with particular shapes without producing a distinct error type. We test for this with a per-model elastic-net analysis over 21 query-level features across 13 models (Appendix I); the result is null. No feature has a stable signed effect across models, and the one feature with a non-zero pooled coefficient (ps_has_scalar_subquery) shows substantial between-model heterogeneity ($I^2 = 66\%$). Per-model preferences exist but dis-

agree, consistent with model-internal factors rather than a shared structural cause.

7 Conclusion

We argued that SQL’s role as the default generation target for text-to-database systems is a legacy assumption worth revisiting. Generating a dialect-agnostic Calcite plan and letting a deterministic compiler handle dialect adaptation restores cross-dialect portability, shrinking the lower-bound accuracy drop from 8–17 pp under SQL to at most 2.3 pp across every model we evaluated, including a LoRA-tuned Ministral-3-8B. We also introduced DF-Match, a question-aware result-set comparator that fixes the cross-dialect failure modes of BIRD-EA and is needed to evaluate any text-to-SQL system fairly across backends.

Scope and outlook. Our evaluation covers four open-source backends (SQLite, PostgreSQL, MySQL, ClickHouse); extending the compiler to Snowflake or BigQuery, whose operator surfaces diverge further from Calcite’s core, is a natural next step, as is a broader sweep across model families and sizes. Beyond portability, the query plan format opens several axes worth studying: *extensibility*, since Calcite’s operator algebra accepts new operators (e.g., semantic operations such as embedding-based retrieval or LLM-judged predicates) without changing the generation interface; *constrained decoding* to eliminate the residual malformed-JSON failures; *test-time scaling* via verifier-guided sampling over a typed operator tree; and *interpretability*, since an operator tree is directly visualizable. We leave these to future work.

Limitations

We discuss limitations along four axes.

Backend coverage. Our compiler and evaluation cover four open-source backends (SQLite, PostgreSQL, MySQL, ClickHouse). Cloud warehouses such as Snowflake and BigQuery, whose operator surfaces diverge further from SQLite’s syntax, are not evaluated.

Benchmark coverage. We evaluate on the dev splits of BIRD and Spider, both of which ship gold queries in SQLite and are English-only. Hidden test sets are not publicly accessible, so we cannot rule out dev-set contamination in the reported numbers. Cross-lingual text-to-SQL is out of scope.

DF-Match validity. The conservatism argument in Appendix H assumes a correct question classification; misclassifying an ordered question as `not_ordered` weakens the comparison to a set check and can introduce false positives, with the empirical rate bounded by the classifier accuracy reported in §6.2. The human-agreement study in Appendix F is single-labeler and drawn from the disagreement set rather than i.i.d. from BIRD, so the reported 96.1% should be read as a directional rather than population estimate.

Best-dialect accuracy. Restoring portability is not free at the top of the SQLite-only leaderboard. For most prompted models, query plans land a few points below SQL on the model’s home dialect (Table 2); the gap closes for capable models above ~ 68 B and inverts under fine-tuning at 8B (§6.4, §6.5). A practitioner whose deployment targets only SQLite, and who is willing to accept the cross-dialect fragility, can still extract slightly higher peak accuracy from direct SQL generation with a sub-capability-threshold model.

Mitigations not evaluated. We hypothesize but do not measure that constrained decoding or resampling would close the residual malformed-JSON surplus reported in §6.6; we leave this to future work along with the broader directions discussed in the conclusion.

References

- Haritha Ananthakrishnan, Harsha Kokel, Kelsey Sikes, Debarun Bhattacharjya, Michael Katz, Shirin Sohrabi, and Kavitha Srinivas. 2026. Querygym: Step-by-step interaction with relational databases. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 41544–41546.
- Apache Software Foundation. 2024. Apache calcite: A dynamic data management framework. <https://calcite.apache.org/>.
- Benjamin G. Ascoli, Yasoda Sai Ram Kandikonda, and Jinho D. Choi. 2024. ETM: Modern insights into perspective on text-to-SQL evaluation in the age of large language models. arXiv preprint arXiv:2407.07313.
- Edmon Begoli, Jesús Camacho-Rodríguez, Julian Hyde, Michael J Mior, and Daniel Lemire. 2018. Apache calcite: A foundational framework for optimized query processing over heterogeneous data sources. *Proceedings of the 2018 International Conference on Management of Data*, pages 221–230.
- dltHub. 2022. dlt: the open-source Python library for data loading. <https://github.com/dlt-hub/dlt>. Accessed 2026-05-11.

- Li Dong and Mirella Lapata. 2016. Language to logical form with neural attention. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 33–43.
- Ben Eyal, Amir Bachar, Ophir Haroche, and Michael Elhadad. 2023. Semantic parsing for complex data retrieval: Targeting query plans vs. SQL for no-code access to relational databases. *arXiv preprint arXiv:2312.14798*.
- Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-sql empowered by large language models: A benchmark evaluation. *Proceedings of the VLDB Endowment*, 17(5):1132–1145.
- Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-constrained decoding for structured nlp tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952.
- Jiaqi Guo, Zecheng Zhan, Yan Gao, Yan Xiao, Jian-Guang Lou, Ting Liu, and Dongmei Zhang. 2019. Towards complex text-to-SQL in cross-domain database with intermediate representation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4524–4535.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Liang Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.
- Heegyu Kim, Taeyang Jeon, Seunghwan Choi, Seungtaek Choi, and Hyunsouk Cho. 2024. FLEX: Expert-level false-less EXecution metric for reliable text-to-SQL benchmark. *arXiv preprint arXiv:2409.19014*.
- Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, et al. 2025. Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. In *International Conference on Learning Representations*.
- Haoyang Li, Shang Wu, Xiaokang Zhang, Xinmei Huang, Jing Zhang, Fuxin Jiang, Shuai Wang, Tieying Zhang, Jianjun Chen, Rui Shi, et al. 2025. Omnisql: Synthesizing high-quality text-to-sql data at scale. *arXiv preprint arXiv:2503.02240*.
- Haoyang Li, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, and Hong Chen. 2024a. Codes: Towards building open-source language models for text-to-sql. *Proceedings of the ACM on Management of Data*, 2(3):1–28.
- Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2024b. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems*, 36.
- Zhisheng Lin, Yifu Liu, Zhiling Luo, Jinyang Gao, and Yu Li. 2024. Momq: Mixture-of-experts enhances multi-dialect query generation across relational and non-relational databases. *arXiv preprint arXiv:2410.18406*.
- Toby Mao. 2023. Sqlglot: A python sql parser, transpiler, optimizer, and engine. <https://github.com/tobymao/sqlglot>.
- Arjun Panickssery, Samuel R Bowman, and Shi Feng. 2024. Llm evaluators recognize and favor their own generations. *Advances in Neural Information Processing Systems*, 37:68772–68802.
- Yotam Perlitz, Elad Venezian, Corentin Royer, Francesco Fusco, and Andrea Giovannini. 2026. Polysql: Scaling text-to-sql evaluation across sql dialects via automated backend isomorphism. *arXiv preprint arXiv:2605.07796*.
- Mohammadreza Pourreza and Davood Rafiei. 2024. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *Advances in Neural Information Processing Systems*, 36.
- Mohammadreza Pourreza, Ruoxi Sun, Hailong Li, Lesly Miculicich, Tomas Pfister, and Sercan O Arik. 2024. Sql-gen: Bridging the dialect gap for text-to-sql via synthetic data and model merging. *arXiv preprint arXiv:2408.12733*.
- Maxim Rabinovich, Mitchell Stern, and Dan Klein. 2017. Abstract syntax networks for code generation and semantic parsing. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1139–1149.
- Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. Picard: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 conference on empirical methods in natural language processing*, pages 9895–9901.
- Jie Shi, Xi Cao, Bo Xu, Jiaqing Liang, Yanghua Xiao, Jia Chen, Peng Wang, and Wei Wang. 2025. Dialect-sql: An adaptive framework for bridging the dialect gap in text-to-sql. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 3604–3619.
- subtrait io. 2021. Subtrait: Cross-language serialization for relational algebra. <https://github.com/subtrait-io/subtrait>.
- Shayan Talaei, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. 2024. Chess: Contextual harnessing for efficient sql synthesis. *arXiv preprint arXiv:2405.16755*.

- Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2020. Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, pages 7567–7578.
- Pengcheng Yin and Graham Neubig. 2018. Tranx: A transition-based neural abstract syntax parser for semantic parsing and code generation. In *Proceedings of the 2018 conference on empirical methods in natural language processing: System demonstrations*, pages 7–12.
- Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 3911–3921.
- John M Zelle and Raymond J Mooney. 1996. Learning to parse database queries using inductive logic programming. In *Proceedings of the national conference on artificial intelligence*, pages 1050–1055.
- Jipeng Zhang, Haolin Yang, Kehao Miao, Ruiyuan Zhang, Renjie Pi, Jiahui Gao, and Xiaofang Zhou. 2025. Exesql: Self-taught text-to-sql models with execution-driven bootstrapping for sql dialects. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 24305–24326.
- Xiang Zhang, Hongming Xu, Le Zhou, Wei Zhou, Xuanhe Zhou, Guoliang Li, Yuyu Luo, Changdong Liu, Guorun Chen, Jiang Liao, and Fan Wu. 2026. Dial: A knowledge-grounded dialect-specific NL2SQL system. arXiv preprint arXiv:2603.07449.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623.
- Ruiqi Zhong, Tao Yu, and Dan Klein. 2020. Semantic evaluation for text-to-SQL with distilled test suites. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 396–411.

A Full Results Table

Tables 2 and ?? report per-dialect execution accuracy for all models on BIRD and Spider respectively. The lower-bound accuracy drop for SQL models is driven by ClickHouse and PostgreSQL, while MySQL accuracy remains close to SQLite across all models. Query plan generation shrinks the drop to at most 2.3 pp (portability ≥ 0.96) across every model, including fine-tuned Ministral-3-8B.

Both tables also report the mean accuracy across the four dialects and, on each model’s query plan row, the query plan–SQL difference in best-dialect (ΔPeak) and worst-dialect (ΔWorst) accuracy, so that the peak-accuracy cost of the switch is legible next to the portability gain rather than hidden behind a ratio.

B Spider Figures

The same trends seen on BIRD hold on Spider. Figure 6 shows portability vs. model size: the query plan cloud sits above the SQL cloud at every scale, with neither cloud trending with size. Figure 7 shows lower-bound dialect accuracy: the query plan trend rises with scale and crosses SQL near 62B, slightly below the BIRD crossover at 68B but consistent with the same overall picture.

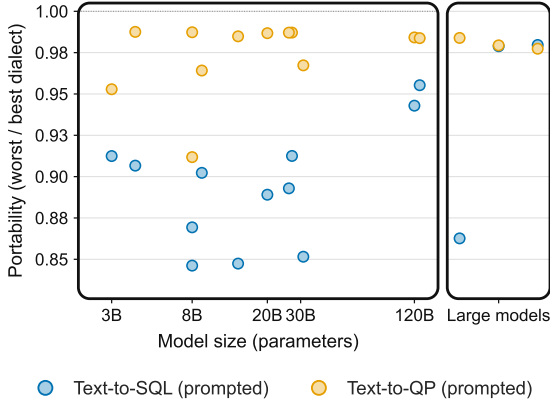


Figure 6: Portability (lower-bound / best-dialect accuracy ratio) vs. model size on Spider. Same encoding as Figure 1.

C Training Details

We fine-tune Ministral-3-8B on 1M examples drawn from SynSQL and BIRD-train (§5.4). The SQL and query plan runs use identical data, optimizer, schedule, and compute budget; the only difference is the supervision target.

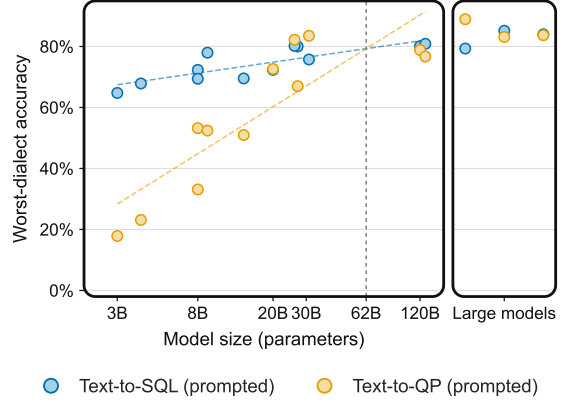


Figure 7: Lower-bound dialect accuracy on Spider vs. model size. Same encoding as Figure 3.

LoRA configuration.

- Rank: 64
- α : 16 (fixed; *not* $2 \cdot \text{rank}$)
- Dropout: 0.05
- Target modules: all linear layers
- Trainable parameters: $\approx 200\text{M}$

Optimization.

- Optimizer: AdamW
- Learning rate: 1×10^{-4}
- Schedule: cosine decay with linear warmup
- Epochs: 1
- Batch size: 64

Software. Fine-tuning uses Hugging Face transformers³ with peft⁴ for the LoRA adapters (Hu et al., 2022). Inference for all open-weight models is served with vLLM⁵. The conversion pipeline uses Apache Calcite (Apache Software Foundation, 2024) and SQLGlot (Mao, 2023) (§4.1), and the cross-dialect database migration uses DLT (dltHub, 2022) (§5.2).

Hardware. Each run is trained on $4 \times \text{NVIDIA A100 GPUs}$. The SQL and query plan runs are given an identical compute budget, so the comparison in §6.5 is matched by construction.

³<https://github.com/huggingface/transformers>

⁴<https://github.com/huggingface/peft>

⁵<https://github.com/vllm-project/vllm>

Model	Target	BIRD-dev					Spider-dev				
		SQLite	PG	MySQL	CH	Port.	SQLite	PG	MySQL	CH	Port.
<i>Small</i>											
Ministral 3-8B [†]	SQL	61.2	52.4	57.4	50.3	82.1	85.3	78.3	81.5	77.6	90.9
Ministral 3-8B [†]	QP	64.5	64.2	65.2	63.5	97.3	84.1	83.2	84.5	82.9	98.2
SQLCoder	SQL	39.1	29.3	35.1	27.6	70.6	66.0	18.4	50.3	18.2	27.5
OmniSQL	SQL	62.1	54.9	58.0	48.7	78.5	86.0	76.1	83.0	76.5	88.5
Qwen 3.5 9B	SQL	66.6	58.7	62.5	55.5	83.4	86.4	79.1	81.9	78.0	90.2
Qwen 3.5 9B	QP	26.8	26.5	26.9	26.7	98.3	53.1	52.4	52.9	54.4	96.4
Gemma 4 E2B	SQL	46.4	41.9	42.7	38.2	82.3	74.8	67.9	69.9	68.9	90.7
Gemma 4 E2B	QP	10.0	10.0	9.9	9.9	99.4	23.3	23.1	23.4	23.2	98.8
Gemma 4 E4B	SQL	58.8	51.9	52.4	47.3	80.4	83.2	72.3	75.7	73.4	86.9
Gemma 4 E4B	QP	27.4	27.6	27.7	27.8	98.6	53.9	53.2	53.4	53.4	98.7
Ministral 3-3B	SQL	43.4	39.8	43.2	34.9	80.6	71.0	66.1	69.7	64.8	91.2
Ministral 3-3B	QP	11.8	11.8	11.9	12.1	97.3	17.9	18.7	18.0	18.0	95.3
Ministral 3-8B	SQL	53.3	49.7	49.9	38.4	72.0	82.0	74.7	80.8	69.4	84.6
Ministral 3-8B	QP	15.7	16.1	16.1	16.0	97.2	33.8	36.3	33.1	33.6	91.2
Ministral 3-14B	SQL	54.4	51.4	52.9	37.4	68.7	82.0	77.6	81.0	69.5	84.7
Ministral 3-14B	QP	18.3	18.7	19.1	18.4	95.9	51.8	51.0	51.6	51.5	98.5
<i>Medium</i>											
GPT OSS 20B	SQL	58.1	56.7	55.1	48.6	83.5	80.3	76.0	81.4	72.3	88.9
GPT OSS 20B	QP	48.5	48.8	49.4	48.9	98.3	73.6	72.6	73.4	73.1	98.7
Qwen 3.5 27B	SQL	69.0	65.1	67.0	59.9	86.9	87.7	83.0	80.0	82.0	91.3
Qwen 3.5 27B	QP	39.5	39.1	39.8	39.4	98.2	67.9	67.0	67.8	67.0	98.7
Gemma 4 26B	SQL	67.8	62.5	62.7	56.2	82.9	89.8	80.2	80.3	83.2	89.3
Gemma 4 26B	QP	54.6	54.3	54.8	54.6	99.1	83.3	82.3	83.1	82.2	98.7
Gemma 4 31B	SQL	70.6	64.8	66.8	58.9	83.4	88.9	75.7	78.4	81.6	85.2
Gemma 4 31B	QP	63.9	64.2	63.7	63.2	98.5	86.3	84.6	84.7	83.5	96.7
<i>Large</i>											
GPT OSS 120B	SQL	64.6	61.1	61.3	55.1	85.3	84.9	80.4	84.9	80.1	94.3
GPT OSS 120B	QP	58.7	58.6	59.8	58.2	97.4	79.8	79.1	80.1	78.8	98.4
Mistral Medium 35	SQL	63.1	59.2	60.1	51.8	82.0	84.7	80.9	84.7	81.3	95.5
Mistral Medium 35	QP	55.9	55.5	56.1	55.3	98.5	78.0	76.7	77.3	76.7	98.4
Claude Sonnet 4.6	SQL	69.4	65.8	67.2	60.1	86.6	87.0	86.1	86.6	85.1	97.9
Claude Sonnet 4.6	QP	64.9	64.3	64.9	63.9	98.4	84.9	83.4	84.1	83.1	97.9
GPT 5.5	SQL	67.3	65.8	65.2	58.0	86.1	85.3	84.0	85.7	85.0	98.0
GPT 5.5	QP	64.9	64.1	63.8	62.6	96.4	85.6	84.9	84.9	83.7	97.7
Gemini 3.1 Pro	SQL	73.2	69.6	70.0	63.1	86.2	91.9	79.3	81.6	88.5	86.3
Gemini 3.1 Pro	QP	68.2	67.5	68.8	67.1	97.4	90.4	89.2	90.1	88.9	98.4

Table 2: Per-dialect execution accuracy (%) on BIRD-dev and Spider-dev under DF-Match; all prompted runs use greedy decoding (single sample). PG = PostgreSQL, CH = ClickHouse. Port. = portability, defined as worst-dialect / best-dialect accuracy (higher = more portable; 100.0 = identical across dialects). [†] LoRA fine-tuned.

D LLM-as-Judge Prompt

Error categorization uses Claude Haiku 4.5 at temperature 0. The input always uses SQLite queries: for query plan models, the predicted plan is converted back to SQLite before classification.

Categories.

- **Schema Linking:** Wrong table or column name, hallucinated schema element.
- **Filtering / Logic:** Wrong WHERE predicate, JOIN condition, subquery logic, or conditional expression.
- **Aggregation / Grouping:** Wrong aggregation function, incorrect GROUP BY or HAVING

clause.

- **Ordering / Ranking:** Wrong ORDER BY column or direction, wrong LIMIT/OFFSET, RANK vs. ROW_NUMBER on ties; also covers structural variants for top-1 selection (e.g. MAX subquery vs. ORDER BY LIMIT 1).
- **Dialect Syntax:** Syntax error, dialect-specific function not valid in SQLite, malformed output.
- **Other:** Genuinely unclassifiable, or ambiguous question where multiple structurally incomparable SQL queries are defensible.

User prompt template.

```

Question: {question}

Gold SQL:
{gold_sql}

Predicted SQL:
{pred_sql}

Error message (if any): {error_msg}

```

The system prompt includes one few-shot example per category to anchor the classifier. The full system prompt is available in the released code.

E Datasets, Licenses, and Intended Use

We use three publicly released datasets, each within its intended research use.

BIRD (Li et al., 2024b). Released under CC BY-SA 4.0. We use the train split (9,428 question–SQL pairs over 69 databases) for fine-tuning and the dev split (1,534 questions over 11 databases) for evaluation. The hidden test set is not publicly available.

Spider (Yu et al., 2018). Released under CC BY-SA 4.0. We use the dev split (1,034 questions) for evaluation only. Spider is intended as a complex, cross-domain semantic-parsing and text-to-SQL benchmark; cross-dialect evaluation, which is what we use it for, is consistent with that purpose.

SynSQL-2.5M (Li et al., 2025). Released under Apache 2.0. The full dataset contains 2,544,390 (*database, question, SQL, chain-of-thought*) quads over 16,583 synthetic databases. We sample 1M examples from this corpus, mixed with BIRD–train, as the fine-tuning source described in §5.4. We do not redistribute or modify the underlying data; the full corpus remains available from its original release.

All three datasets are English-only and target SQLite at source. Our cross-dialect evaluation derives PostgreSQL, MySQL, and ClickHouse copies via schema-level ETL from the released SQLite databases (§5.2); the natural-language questions and gold queries are unchanged.

F DF-Match Validation Details

We validate DF-Match on three fronts: the accuracy of the question classifier that drives its per-class comparison strategy, agreement with human judgment on cases where DF-Match disagrees with BIRD-EA, and the effect of the metric choice on the reported reliability of the conversion pipeline itself.

Classifier validation. DF-Match’s comparison strategy hinges on correctly classifying each question along the ordering and row-count axes (§4.2). We validate the LLM classifier on 103 BIRD questions annotated by hand along both axes, reporting balanced accuracy per axis: **93.44%** on ordering sensitivity and **99.50%** on row-count sensitivity. The two axes are decided independently. Questions misclassified as `not_ordered` fall back to BIRD-EA-like set comparison, which is strictly more permissive than the ordered variants and is therefore the dominant residual source of false positives in DF-Match (see Appendix H); the empirical false-positive rate is bounded by the classifier’s per-axis accuracy reported above.

Human agreement on disagreement cases. We sampled 103 BIRD predictions on which DF-Match and BIRD-EA reach opposite verdicts, and had a human annotator decide which verdict matches the intended answer to the natural-language question. Two annotators contributed, both authors with strong SQL proficiency, with each example labelled by exactly one of them. The instructions target semantic correctness rather than raw result equality: they state that different SQL queries can produce the same correct answer, and that differing row order where the question does not require one, differing column names, and harmless extra columns should all be accepted. As a control against favouring our own representation, annotators never saw the query plan: every prediction was compiled to SQLite before display, so labellers were blind to both the generation target and the dialect it came from. Table 3 shows that humans side with DF-Match in 96.1% of cases and with BIRD-EA in 3.9%. A McNemar test on the contingency table gives $\chi^2 = 85.79$, $p \approx 2 \times 10^{-20}$, effect size 0.92 (large). Concretely, on the set where the two metrics disagree, the disagreement is a BIRD-EA misclassification roughly $25\times$ more often than a DF-Match misclassification. We note two limitations of this study: annotations are single-labeler per example (no inter-annotator agreement reported), and the sample is drawn from the disagreement set rather than i.i.d. from BIRD.

Annotator instructions. The annotator received the instructions reproduced below alongside a per-example interface displaying the natural-language question, the gold SQL (left), the predicted SQL (right), and both result tables. Each example required one of three correctness verdicts (CORRECT,

Metric	Count	%
DF-Match	99	96.1
BIRD-EA	4	3.9

Table 3: Human agreement on 103 BIRD cases where DF-Match and BIRD-EA disagree. McNemar $\chi^2 = 85.79$, $p = 2 \times 10^{-20}$.

UNSURE, INCORRECT), with two optional flags for downstream filtering (GOLD-IS-WRONG, OTHER-ISSUE) and an optional free-text note required only when a flag was set.

Task. Evaluate whether the predicted SQL correctly answers the natural-language question by comparing its result to the gold SQL’s result. Focus on *semantic correctness*: different SQL queries can produce the same correct answer.

Verdicts.

- SEMANTICALLY CORRECT. The predicted result answers the question correctly, even if the SQL itself differs. Same data in both result tables; same values, possibly in a different order if the question does not specify one; column names may differ; harmless extra columns are tolerated.
- UNSURE / PARTIALLY CORRECT. Use sparingly, only for rare edge cases: results are correct but the logic to reach them is wrong, or correctness is genuinely uncertain. Most cases should resolve to CORRECT or INCORRECT.
- INCORRECT. The predicted SQL is clearly wrong: wrong logic, missing or extra rows, or wrong columns returned.

Flags (optional).

- GOLD-IS-WRONG: the gold reference itself appears incorrect.
- OTHER-ISSUE: ambiguous question, schema problem, or technical issue.

A free-text note field is shown but only required to be filled when a flag is set.

Pipeline reliability under both metrics. Table 4 re-runs the round-trip analysis of Section 6.1 on BIRD under BIRD-EA alongside DF-Match. Because the round-trip executes the *same* logical query on the *same* data in two different dialects, any “failure” under BIRD-EA is either a genuine

Metric	SQLite	PG	MySQL	CH
BIRD-EA	83.1	81.2	79.7	80.6
DF-Match	97.1	95.4	95.6	94.7
Gap	+14.0	+14.2	+15.9	+14.1

Table 4: Round-trip conversion reliability (%) on BIRD under DF-Match vs. BIRD-EA. PG = PostgreSQL, CH = ClickHouse. The consistent 14–16 pp gap per dialect is a metric artefact, not a pipeline defect.

conversion defect or a metric artefact. The gap between the two rows per dialect is therefore a direct measurement of how much of the reported unreliability is metric-induced. Under BIRD-EA, apparent reliability drops by 13–16 pp on every dialect including SQLite, with no corresponding change in the underlying conversion: the drop is explained by cross-dialect row-order nondeterminism in grouped results and by boundary-tie multiplicity that BIRD-EA has no mechanism to tolerate. This result is a cautionary tale in its own right: any cross-dialect evaluation conducted under BIRD-EA will substantially underreport the pipeline or model it is evaluating.

G DF-Match pseudocode

The per-class dispatch of DF-Match is summarized below. The full implementation, including column-mapping discovery, tie detection, and LIMIT boundary-alternative extraction, is released as part of the code accompanying this paper.

```
function DF-MATCH(pred, gold, class):
  pred <- MAP-COLUMNS(pred, gold)
  if pred is None: # gold column unmappable
    return false
  if class == "not_ordered":
    return MULTISSET-EQUAL(pred, gold)
  if class == "ordered_fixed":
    if |pred| != |gold|:
      return false
    ties <- DETECT-TIES(gold.order_keys)
    return POSITIONAL-MATCH(
      pred, gold, tie_groups=ties)
  # class == "ordered_not_fixed"
  if |pred| < |gold|:
    return false
  pred <- pred[:|gold|]
  alts <- BOUNDARY-ALTERNATIVES(
    gold.order_keys, gold.limit)
  return ORDERED-MATCH(pred, gold,
    boundary_alts=alts)
```

POSITIONAL-MATCH accepts any permutation of rows that share a tie group (rows whose ORDER BY keys are equal up to $\varepsilon = 10^{-9}$). ORDERED-MATCH enforces exact ordering for rows before the LIMIT

cut and accepts any row at or beyond the cut whose key values match the pre-computed boundary alternatives. All numeric comparisons are ε -tolerant.

H DF-Match conservatism: proof sketch

We sketch why DF-Match produces no false positives, given a correct question classification (§4.2). A false positive is a verdict of “correct” for a predicted dataframe $\hat{T}$ that does not match the gold T^* under the question’s intended semantics. We argue per class.

not_ordered. Comparison reduces to a multiset (or set, under DISTINCT-tolerance) equality on rows after column mapping. Column mapping requires every gold column to be matched to some predicted column with the same distinct values; an unmappable gold column rejects $\hat{T}$ outright. Given a valid mapping, the multiset/set check accepts only when every gold row has a matching predicted row (and, in the strict variant, vice versa). $\hat{T}$ cannot contain a row absent from T^* without breaking the equality, so the verdict cannot be “correct” on a mismatched answer.

ordered_fixed. Cardinality must match exactly, and rows are compared positionally, with permutations permitted only *within tie groups*. Tie groups are computed from the gold ordering keys (Section 4.2): two rows share a tie group iff their gold-side ORDER BY expressions are equal (up to ε). A predicted row with a different ordering-key value cannot fall into the same tie group as the gold row at that position, and so cannot be permuted into place. Any divergence in the answer set or in the order of distinct-key rows is rejected.

ordered_not_fixed. Predicted output is truncated to $|T^*|$ rows; if $|\hat{T}| < |T^*|$, the verdict is “incorrect”. The truncated prefix is then compared as in ordered_fixed, with one extension: a row at the LIMIT boundary may be replaced by a row from the boundary-alternatives query, which enumerates exactly those rows the gold query *could have legitimately returned* at the cut (Section 4.2). By construction, the alternatives set contains only rows whose ordering-key values equal the gold row’s at the boundary, so accepting one in place of the gold row preserves the gold query’s semantics.

Conservatism caveat. The argument assumes a correct question classification and a faithfully extractable order-key/boundary-alternatives query

from the gold SQL. Misclassification of an ordered question as not_ordered weakens the comparison to a set check (strictly more permissive), which can introduce false positives; this is the dominant residual failure mode and is bounded empirically by the classifier accuracy reported in §6.2.

I Feature-level meta-analysis: details

This appendix provides the target definition, model list, feature definitions, pooling protocol, and full results behind §6.6.

Target. We define qp_beats_sql on *divergent* queries, those where exactly one of SQL and query plan is correct, with qp_beats_sql=1 iff the query plan is the correct one. Restricting to divergent rows isolates the representation effect from query difficulty: queries where both sides agree (both correct, or both wrong) carry no signal about which representation a model prefers. As a sanity check, a difficulty-only baseline (predicting per-model task accuracy from the same features) achieves cross-validated AUC 0.50–0.65, well below the 0.74–0.87 the same features achieve when targeting individual SQL and query plan outcomes; qp_beats_sql is therefore not just a re-encoded difficulty target.

Models. The meta-analysis pools per-model coefficients across 13 models spanning 6 families and four size tiers: Ministral-3-3B, Ministral-3-8B (prompted and LoRA fine-tuned on SQL), Gemma 4 E4B, Qwen 3.5 9B, GPT-OSS 20B, Qwen 3.5 27B, Gemma 4 31B, Qwen 3.5 122B, Mistral Medium, GPT-OSS 120B, Claude Sonnet 4.6, GPT-5.5, and Gemini 3.1 Pro.

Features. Twenty-one query-level structural features enter the elastic-net regression, organized into four conceptual clusters (A–D; 17 features) plus two derived families (ps_*, ax_*; 4 features). Names use the prefix conventions sc_* (SQL complexity from a sqlglot parse), ps_* (plan structure from the gold plan tree), qc_* (categorizations of either side), and ax_* (cross-representation asymmetry).

Cluster A, Planning horizon (how far across the SQL text dependent information appears): qc_scope_cross_clause (column alias used in ORDER BY/GROUP BY/HAVING, or positional ORDER BY); qc_n_forward_refs (count of identifier uses appearing textually before their definition); qc_max_alias_distance (largest character distance between a forward use

and its definition); `qc_type_coercion_level` (0–3 ordinal, combining cast counts and date-function presence); `qc_n_casts` (CAST/TRY_CAST count); `qc_has_date_functions` (any of `strftime`, `date`, `format_date`, `to_char`, etc.); `qc_has_implicit_coercion` (numeric-vs-string comparison in a predicate).

Cluster B, Relational topology (shape of the gold plan tree): `qc_dag_max_width` (max number of nodes at any depth); `qc_max_join_depth` (longest JOIN→JOIN chain).

Cluster C, Semantic complexity (what the query computes): `qc_n_reasoning_steps` (non-leaf plan node count); `qc_literal_diversity_level` (0–3 ordinal over literal count, type diversity, date and pattern presence); `qc_n_literals`; `qc_has_pattern_literal` (string literal containing % or _); `qc_category_ordered_fixed` and `qc_category_ordered_not_fixed` (two one-hot indicators expanding the result-shape contract from §4.2; `not_ordered` is the reference category).

Cluster D, Format gap (structural differences between SQL and plan): `qc_locality_gap` (plan locality – SQL locality, where locality is the fraction of identifier uses within a short distance of their definition); `qc_n_ambiguous_refs` (count of unqualified column references in multi-table queries).

Plan structure (`ps_*`): `ps_max_depth` (longest root-to-leaf path); `ps_has_scalar_subquery` (any plan node with a nested scalar-subquery operator).

Asymmetry (`ax_*`): `ax_length_ratio` ($(ps_n_nodes / \max(sc_n_tokens, 1))$); `ax_length_diff` ($ps_n_nodes - sc_n_tokens$).

The full `sc_*` family is computed but excluded from the regression: its signal is captured by the `ps_*` and `qc_*` plan-side features plus the empirical difficulty confounder.

Pooling. For each model and each binary target we fit an elastic-net logistic regression on the 21 features above with bootstrap resampling. Per-model coefficients $\hat{\beta}_m$ and standard errors se_m are pooled via inverse-variance-weighted fixed-effect meta-analysis: $\hat{\beta} = \sum_m w_m \hat{\beta}_m / \sum_m w_m$ with $w_m = 1/se_m^2$. We report Cochran’s I^2 as the heterogeneity diagnostic, the proportion of total variation in $\hat{\beta}_m$ attributable to between-model differences rather than sampling error. High I^2 indicates a feature whose effect is model-specific and pools to near-zero by cancellation; low I^2 with

a non-zero pooled estimate indicates a universal effect.

Results. Of 21 features, only one has a pooled CI excluding zero: `ps_has_scalar_subquery` ($\hat{\beta} = -0.04$, 95% CI $[-0.07, -0.004]$, $I^2 = 66\%$). All others average to near-zero with inconsistent per-model signs. Per-model preferences do exist but disagree across models (e.g., Claude Sonnet 4.6 favors SQL on ordered queries, while GPT-OSS 120B favors query plans with scalar subqueries), consistent with model-internal factors such as tokenization and format familiarity that query-level features cannot capture.

J Inference Overhead and Compiler Extension Cost

Token cost of the representation. Query plans are more verbose than the SQL they replace. Measured over BIRD, the emitted plan averages 227 output tokens against 58 for the corresponding SQL query, so the representation itself costs about $3.9\times$ the tokens of the query it replaces.

This artefact cost is not the whole generation cost. For both targets, most output tokens are reasoning rather than the query artefact: of roughly 319 output tokens per SQL generation only about 58 are the query, and of roughly 1685 per query plan generation only about 227 are the plan. End to end the ratio is therefore larger than $3.9\times$, because both formats carry a long shared reasoning trace that is not specific to either representation. Reducing reasoning verbosity shrinks the absolute gap for both targets, since the representation accounts for an absolute difference of roughly 170 tokens.

We report output tokens rather than wall-clock latency. Tokens are the hardware-independent quantity; latency depends on serving stack, batch size, and hardware, and we did not measure it under controlled conditions.

Cost of supporting a new dialect. Adding a backend takes roughly one engineer-day, because we do not implement a dialect from scratch. Calcite ships approximately 40 dialects, including Snowflake and BigQuery, and the work is to align their semantics with the conventions our pipeline assumes: NULL ordering, date handling, cast behaviour, and identifier case sensitivity (§4.1). In our experience the larger cost was not the dialect but the benchmark environment, that is migrating and hosting four parallel databases with identical

contents. That is a cost of *measuring* portability, not of deploying it: a production system targets one backend it already runs.